

CrewAI / AutoGen 개념 비교 및 코드 예시

1. 개요

멀티에이전트 시스템에서는 여러 Agent가 서로 다른 역할을 수행하면서 하나의 문제를 해결한다.

앞에서 살펴본 LangGraph 방식에서는 Orchestrator, Sub-agent, Sequential, Hierarchical, Message Passing 등의 구조를 Graph의 Node와 Routing 코드로 직접 구현하였다.

CrewAI와 AutoGen은 이러한 멀티에이전트 협업 패턴을 미리 제공하는 고수준 프레임워크이다. 따라서 Agent 간의 실행 흐름을 처음부터 직접 구현하지 않고, 역할이나 대화 규칙을 정의하여 멀티에이전트 시스템을 구성할 수 있다.

두 프레임워크의 가장 큰 차이는 다음과 같이 이해할 수 있다.

CrewAI

→ 역할 (Role) 과 작업 (Task) 을 중심으로 협업

AutoGen

→ Agent 간 대화 (Conversation)와 메시지 교환을 중심으로 협업

2. CrewAI와 AutoGen의 핵심 비교

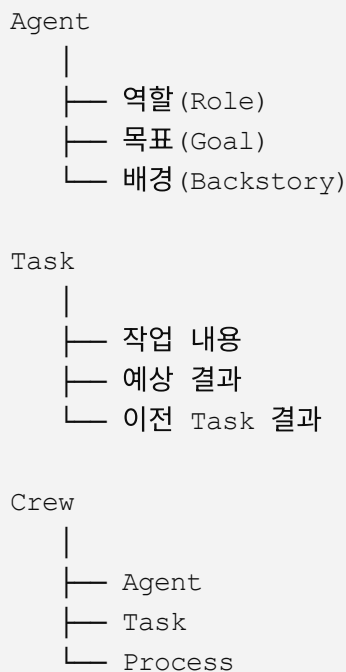
구분	CrewAI	AutoGen
핵심 패러다임	역할 기반 협업	대화 기반 협업
주요 구성	Agent + Task + Crew	Agent + Conversation + GroupChat
실행 방식	Sequential / Hierarchical	GroupChatManager 기반 대화
Agent 정의	role, goal, backstory	system_message
작업 정의	Task	메시지와 대화
Agent 간 결과 전달	Task의 context	Agent 간 메시지
Manager 역할	hierarchical process	GroupChatManager
Human-in-the-loop	상대적으로 제한적	UserProxyAgent 활용 가능

구분	CrewAI	AutoGen
적합한 작업	조사 → 작성 → 검토 등 명확한 Pipeline	토론, 브레인스토밍, 대화형 협업
추상화 수준	높음	높음

두 프레임워크는 앞에서 직접 구현한 멀티에이전트 개념을 더 높은 수준에서 추상화한 것으로 이해하면 된다.

3. CrewAI의 기본 개념

CrewAI는 다음 세 가지 구성 요소를 중심으로 이해하면 된다.



3.1 Agent

Agent는 특정 역할을 담당하는 전문가이다.

주요 속성은 다음과 같다.

```

role
goal
backstory
  
```

예를 들어 시장 조사 Agent는 다음과 같이 정의할 수 있다.

Python

실행됨

```
researcher = Agent(  
    role="시장 조사원",  
    goal="주어진 주제에 대한 핵심 트렌드 3가지를 조사한다",  
    backstory="여러 산업 리포트를 분석해온 경험 많은 시장 조사 전문가다.",  
    llm="gpt-4o-mini",  
    verbose=True,  
)
```

즉, CrewAI에서는 Agent에게 "무엇을 하는 전문가인가"를 명확하게 지정한다.

4. Task

Task는 Agent가 수행해야 하는 구체적인 작업이다.

Python

실행됨

```
research_task = Task(  
    description="'{topic}'에 대한 최신 트렌드 3가지를 조사하고 각각을 한 문장으로 요약하  
    expected_output="트렌드 3개, 각각 한 문장 요약",  
    agent=researcher,  
)
```

여기에서 주요 요소는 다음과 같다.

속성	의미
description	Agent가 수행할 작업
expected_output	예상되는 결과 형식
agent	작업을 담당할 Agent
context	다른 Task의 결과

5. Task 간 결과 전달

CrewAI에서는 Task의 `context` 를 이용하여 이전 Task의 결과를 다음 Task에 전달할 수 있다.

예를 들어 다음과 같은 구조이다.

```
Research Task  
  |  
  | 결과  
  ▼  
Writer Task
```

↓
최종 보고서

코드로 표현하면 다음과 같다.

Python

실행됨

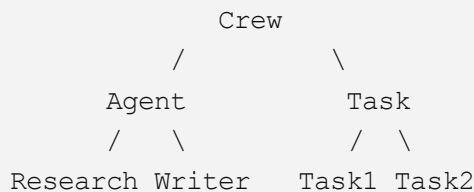
```
write_task = Task(
    description="조사 결과를 바탕으로 경영진 보고용 3문단 요약 보고서를 작성하라.",
    expected_output="3문단으로 구성된 보고서",
    agent=writer,
    context=[research_task],
)
```

즉, `research_task`의 결과가 `write_task`의 입력으로 전달된다.

이는 앞에서 학습한 Orchestrator/Sub-agent 구조에서 하나의 Sub-task 결과를 다음 Sub-task에 전달하는 것과 같은 개념이다.

6. Crew

Crew는 Agent와 Task를 하나의 협업 단위로 묶는다.



예를 들어 다음과 같이 구성할 수 있다.

Python

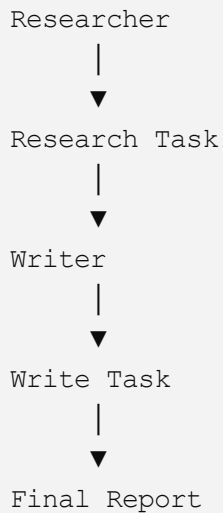
실행됨

```
crew = Crew(
    agents=[researcher, writer],
    tasks=[research_task, write_task],
    process=Process.sequential,
    verbose=True,
)
```

여기에서 `process`가 Agent와 Task의 실행 방식을 결정한다.

7. CrewAI Sequential Process

Sequential은 Task를 정해진 순서대로 실행하는 방식이다.



앞에서 학습한 Sequential Graph와 동일한 개념으로 이해할 수 있다.

Python

실행됨

```
crew = Crew(
    agents=[researcher, writer],
    tasks=[research_task, write_task],
    process=Process.sequential,
    verbose=True,
)
```

실행은 다음과 같이 수행할 수 있다.

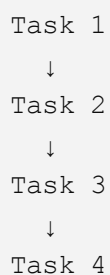
Python

실행됨

```
result = crew.kickoff(
    inputs={"topic": "온디바이스 AI 반도체"}
)

print(result)
```

Sequential 방식에서는 실행 순서가 비교적 명확하다.



따라서 다음과 같은 업무에 적합하다.

시장 조사
↓
자료 분석
↓
보고서 작성
↓
최종 요약

8. CrewAI Hierarchical Process

Hierarchical 방식에서는 Manager 역할을 하는 Agent가 전체 작업을 조정한다.



Sequential 방식과 가장 큰 차이는 실행 순서를 단순히 고정하지 않는다는 점이다.

Manager가 현재 작업 상황을 보고 어떤 Agent에게 작업을 맡길지 결정하는 구조이다.

9. Hierarchical Process 코드 예시

먼저 검토 Agent를 추가한다.

Python

실행됨

```
reviewer = Agent(  
    role="검토자",  
    goal="보고서에 근거 없는 주장이나 어색한 표현이 없는지 검토한다",  
    backstory="사실 확인과 교정에 엄격한 편집자다.",  
    llm="gpt-4o-mini",  
    verbose=True,  
)
```

그 다음 Hierarchical Crew를 구성한다.

Python

실행됨

```
hierarchical_crew = Crew(
    agents=[researcher, writer, reviewer],
    tasks=[research_task, write_task],
    process=Process.hierarchical,
    manager_llm="gpt-4o-mini",
    verbose=True,
)
```

실행은 다음과 같다.

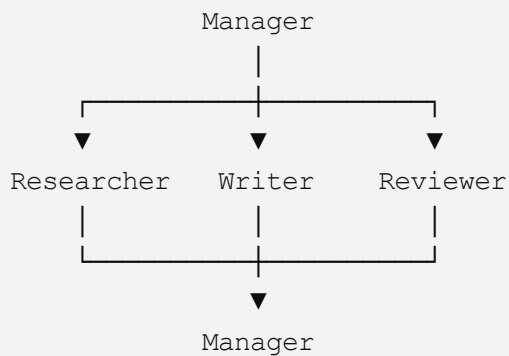
Python

실행됨

```
result = hierarchical_crew.kickoff(
    inputs={"topic": "온디바이스 AI 반도체"}
)

print(result)
```

개념적으로는 다음과 같은 구조이다.



앞에서 LangGraph로 직접 구현했던 `manager_node` 와 유사한 역할을 CrewAI의 Hierarchical Process가 담당한다고 이해하면 된다.

10. CrewAI의 장점

CrewAI의 핵심 장점은 역할과 작업을 비교적 선언적으로 정의할 수 있다는 것이다.

직접 Graph를 구성하면 다음과 같은 요소를 직접 관리해야 한다.

```
Node
Edge
State
Router
Conditional Routing
Loop
```

반면 CrewAI에서는 다음과 같이 역할과 작업을 정의하는 데 집중할 수 있다.

Agent
Task
Crew
Process

따라서 다음과 같은 Pipeline에 적합하다.

조사 Agent
↓
작성 Agent
↓
검토 Agent
↓
최종 결과

11. AutoGen의 기본 개념

AutoGen은 CrewAI와 접근 방식이 다르다.

CrewAI가 Agent의 "역할"을 중심으로 구성한다면 AutoGen은 Agent를 "대화 참여자"로 보는 관점이 강하다.

Agent A
|
| Message
▼
Agent B
|
| Response
▼
Agent A

즉, Agent 간 메시지 교환이 핵심이다.

12. AssistantAgent

AssistantAgent는 LLM을 기반으로 작업을 수행하는 Agent이다.

Agent의 역할은 `system_message`를 통해 지정한다.

Python

```
assistant = autogen.AssistantAgent(  
    name="Researcher",
```

실행됨


```

system_message=(
    "너는 시장 조사 전문가다. "
    "요청받은 주제의 핵심 트렌드를 간결하게 정리해서 답한다."
),
llm_config=llm_config,
)

```

CrewAI와 비교하면 다음과 같다.

```

CrewAI
Agent
├── role
├── goal
└── backstory

AutoGen
AssistantAgent
└── system_message

```

13. UserProxyAgent

UserProxyAgent는 사람 또는 자동화된 실행 주체를 표현할 수 있다.

Python

실행됨

```

user_proxy = autogen.UserProxyAgent(
    name="User",
    human_input_mode="NEVER",
    code_execution_config=False,
    max_consecutive_auto_reply=1,
)

```

특히 `human_input_mode`를 이용하여 사람의 개입 방식을 설정할 수 있다.

예를 들어:

```

사람 개입
↓
Agent
↓
결과 확인
↓
사람 승인
↓
다음 작업

```

과 같은 Human-in-the-loop Workflow를 구성할 수 있다.

14. AutoGen의 기본 대화

두 Agent가 서로 메시지를 주고받는 구조이다.



예를 들어 다음과 같이 시작할 수 있다.

Python

실행됨

```
user_proxy.initiate_chat(
    assistant,
    message="온디바이스 AI 반도체 트렌드 3가지를 알려줘."
)
```

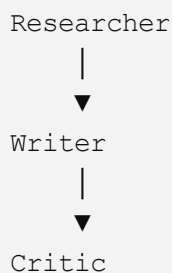
여기에서 핵심은 `initiate_chat()` 이다.

Agent가 작업을 일방적으로 호출하는 것이 아니라 대화가 시작되고 메시지 교환을 통해 작업이 진행된다.

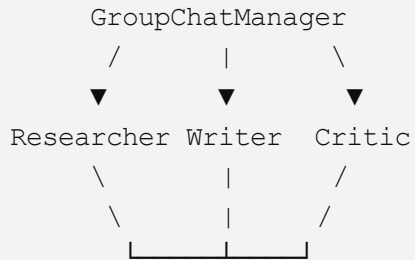
15. GroupChat

Agent가 두 개가 아니라 여러 개라면 GroupChat을 사용할 수 있다.

예를 들어 다음과 같은 세 Agent를 구성할 수 있다.



그러나 실제 대화에서는 단순히 고정된 순서로 실행하는 것이 아니라 현재 상황에 따라 다음 Agent를 선택할 수 있다.



16. GroupChatManager

GroupChatManager는 여러 Agent 중 다음에 발언할 Agent를 결정하는 역할을 수행한다.

예를 들어:

Python

실행됨

```
groupchat = autogen.GroupChat(
    agents=[admin, assistant, writer, critic],
    messages=[],
    max_round=6,
    speaker_selection_method="auto",
)
```

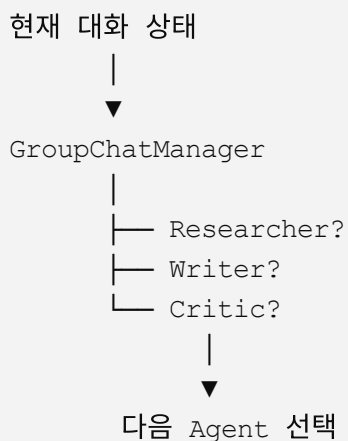
그리고 Manager를 생성한다.

Python

실행됨

```
manager = autogen.GroupChatManager(
    groupchat=groupchat,
    llm_config=llm_config,
)
```

개념적으로는 다음과 같다.



앞에서 LangGraph로 직접 구현했던 `manager_node` 와 유사한 역할을 수행한다.

17. Speaker Selection

GroupChat에서는 다음 발언자를 선택하는 방법을 지정할 수 있다.

대표적인 방식은 다음과 같다.

```
auto
round_robin
random
custom function
```

auto

LLM이 현재 대화 상황을 판단하여 다음 Agent를 선택한다.

Python

```
speaker_selection_method="auto"
```

실행됨

round_robin

Agent를 정해진 순서대로 실행한다.

```
Agent A
↓
Agent B
↓
Agent C
↓
Agent A
```

random

다음 Agent를 무작위 방식으로 선택한다.

custom

사용자가 직접 발언자 선택 규칙을 구현할 수 있다.

18. AutoGen GroupChat 코드 예시

Writer Agent를 정의한다.

Python

실행됨

```
writer = autogen.AssistantAgent(  
    name="Writer",  
    system_message=(  
        "너는 보고서 작성자다. "  
        "Researcher의 조사 내용을 3문단 보고서로 정리한다."  
    ),  
    llm_config=llm_config,  
)
```

Critic Agent를 정의한다.

Python

실행됨

```
critic = autogen.AssistantAgent(  
    name="Critic",  
    system_message=(  
        "너는 검토자다. "  
        "Writer의 보고서에 근거 없는 주장이 있으면 구체적으로 지적하라. "  
        "문제가 없으면 'APPROVED'라고만 답하라."  
    ),  
    llm_config=llm_config,  
)
```

Admin Agent를 정의한다.

Python

실행됨

```
admin = autogen.UserProxyAgent(  
    name="Admin",  
    human_input_mode="NEVER",  
    code_execution_config=False,  
)
```

GroupChat을 구성한다.

Python

실행됨

```
groupchat = autogen.GroupChat(  
    agents=[admin, assistant, writer, critic],  
    messages=[],  
    max_round=6,  
    speaker_selection_method="auto",  
)
```

Manager를 연결한다.

Python

실행됨

```
manager = autogen.GroupChatManager(  
    groupchat=groupchat,  
    llm_config=llm_config,  
)
```

그리고 대화를 시작한다.

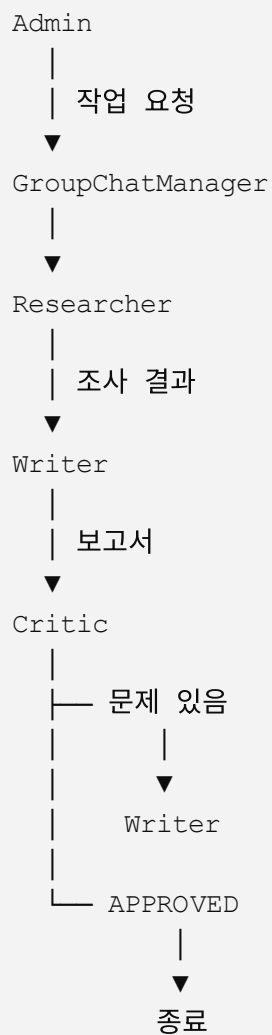
Python

실행됨

```
admin.initiate_chat(  
    manager,  
    message="온디바이스 AI 반도체에 대한 3문단 보고서를 만들어줘."  
)
```

19. AutoGen의 대화형 협업 흐름

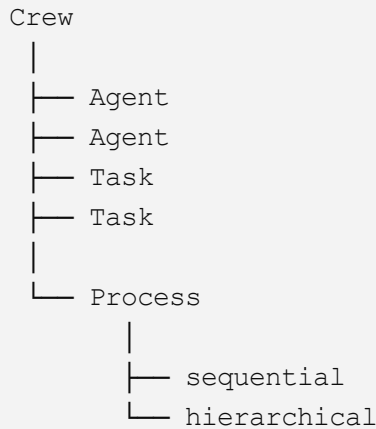
위 구조를 단순화하면 다음과 같다.



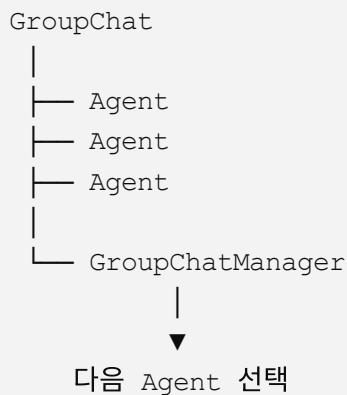
이러한 구조를 사용하면 단순한 순차 실행뿐만 아니라 검토와 재작업을 포함하는 협업 Workflow도 구성할 수 있다.

20. CrewAI와 AutoGen의 실행 구조 비교

CrewAI



AutoGen



따라서 CrewAI는 "업무와 역할"을 중심으로 이해하는 것이 쉽고, AutoGen은 "대화과 메시지"를 중심으로 이해하는 것이 쉽다.

21. LangGraph와의 대응 관계

앞에서 직접 구현한 구조와 연결해서 보면 다음과 같다.

직접 구현한 구조	CrewAI	AutoGen
Sequential StateGraph	Process.sequential	GroupChat + round_robin
Manager Node	Process.hierarchical	GroupChatManager

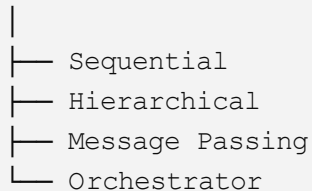
직접 구현한 구조	CrewAI	AutoGen
조건부 Routing	Hierarchical Process	speaker selection
Sub-task 결과 전달	Task의 context	Agent 간 메시지
Agent 간 Message	Task/Context 중심	Conversation
반복 제한	Task 실행 설정	max_round

즉, 지금까지 학습한 개념을 버리고 새로운 프레임워크를 배우는 것이 아니다.

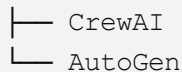
오히려 다음과 같이 연결해서 이해하면 된다.

직접 구현

LangGraph



고수준 프레임워크



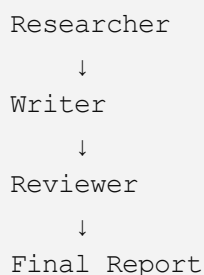
22. CrewAI와 AutoGen의 철학 차이

CrewAI

CrewAI는 다음과 같은 관점에 가깝다.

"각 Agent에게 역할과 작업을 부여하고
전체 Crew가 협업하도록 한다."

따라서 다음과 같은 Workflow에 적합하다.



업무의 역할이 명확한 경우 이해하고 구현하기 쉽다.

AutoGen

AutoGen은 다음과 같은 관점에 가깝다.

"여러 Agent를 대화에 참여시키고
Agent 간 대화를 통해 문제를 해결한다."

따라서 다음과 같은 Workflow에 적합하다.

```
graph TD
    Researcher --> Writer
    Writer --> Critic
    Critic --> Manager
```

Agent 간 자유로운 의견 교환이나 토론이 필요한 경우 유용하다.

23. 어떤 프레임워크가 적합한가

CrewAI가 적합한 경우

업무가 역할별로 명확하다.

```
graph TD
    A[조사 Agent] --> B[작성 Agent]
    B --> C[검토 Agent]
```

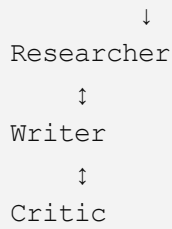
예:

- 시장 조사
- 보고서 작성
- 콘텐츠 제작
- 데이터 분석
- 문서 검토

와 같은 업무이다.

AutoGen이 적합한 경우

여러 Agent가 의견을 주고받아야 한다.



예:

- 브레인스토밍
- 토론
- 코드 검토
- 다중 Agent 문제 해결
- 사람의 승인과 개입이 필요한 Workflow

등이다.

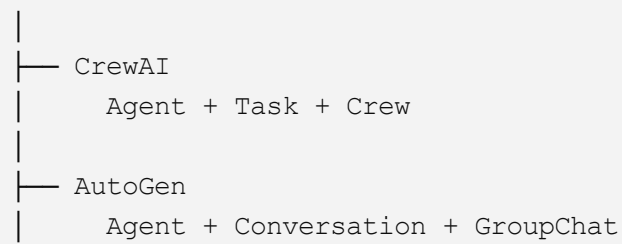
24. CrewAI와 AutoGen의 장단점

항목	CrewAI	AutoGen
학습 관점	역할/Task 중심이라 구조가 명확함	대화 구조를 이해해야 함
Workflow 표현	Pipeline 표현에 유리	대화형 Workflow에 유리
Agent 협업	역할 기반	메시지/대화 기반
동적 실행	Hierarchical로 가능	GroupChat으로 가능
Human-in-the-loop	상대적으로 제한적	강점
자유로운 대화	상대적으로 제한적	강점
정형화된 업무	강점	가능
복잡한 토론	가능	강점

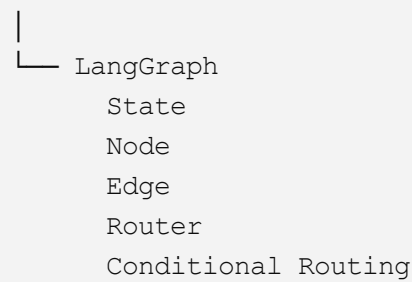
25. LangGraph와 비교

LangGraph는 두 프레임워크와 비교하면 더 낮은 수준에서 Workflow를 직접 제어할 수 있다.

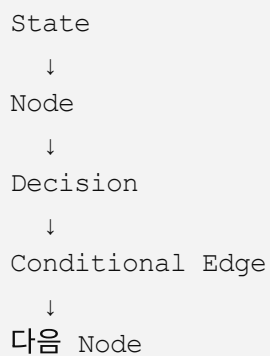
높은 추상화



낮은 추상화



LangGraph에서는 다음을 직접 구현할 수 있다.



따라서 복잡한 Routing 조건이나 세밀한 State 제어가 필요한 경우 유리하다.

반대로 CrewAI와 AutoGen은 이미 만들어진 협업 패턴을 활용하여 빠르게 멀티에이전트 시스템을 구성할 수 있다는 장점이 있다.

26. 실습 환경 구성

제공된 실습 코드에서는 다음과 같은 패키지를 사용한다.

Bash

```
pip install crewai
```

AutoGen 계열 실습에서는 다음과 같은 형태를 사용한다.

Bash

```
pip install pyautogen
```

또는 환경에 따라:

Bash

```
pip install ag2
```

실습 환경에서는 두 프레임워크의 의존성 충돌 가능성을 고려하여 별도의 가상환경을 사용하는 것이 좋다.

또한 패키지를 설치한 후 Jupyter Notebook을 사용하는 경우 커널을 재시작하는 것이 필요할 수 있다.

27. API Key 설정

LLM을 사용하는 경우 API Key를 환경변수로 관리하는 것이 좋다.

예:

Python

실행됨

```
import os
from dotenv import load_dotenv

load_dotenv("C:/env/.env")
```

그리고 환경변수에서 API Key를 가져온다.

Python

실행됨

```
api_key = os.environ["OPENAI_API_KEY"]
```

API Key를 Python 코드에 직접 입력하거나 Notebook에 저장하는 방식은 피하는 것이 좋다.

28. CrewAI 전체 예제

다음은 하나의 완성된 Sequential 예제이다.

Python

실행됨

```
from crewai import Agent, Crew, Process, Task
```

```

researcher = Agent(
    role="시장 조사원",
    goal="주어진 주제에 대한 핵심 트렌드 3가지를 조사한다",
    backstory="여러 산업 리포트를 분석해온 경험 많은 시장 조사 전문가다.",
    llm="gpt-4o-mini",
)

writer = Agent(
    role="보고서 작성자",
    goal="조사 결과를 경영진이 읽기 좋은 보고서로 정리한다",
    backstory="복잡한 조사 내용을 간결하게 정리하는 작가다.",
    llm="gpt-4o-mini",
)

research_task = Task(
    description="'{topic}'에 대한 최신 트렌드 3가지를 조사하라.",
    expected_output="트렌드 3개와 각각의 한 문장 요약",
    agent=researcher,
)

write_task = Task(
    description="조사 결과를 바탕으로 3문단 보고서를 작성하라.",
    expected_output="3문단 보고서",
    agent=writer,
    context=[research_task],
)

crew = Crew(
    agents=[researcher, writer],
    tasks=[research_task, write_task],
    process=Process.sequential,
)

result = crew.kickoff(
    inputs={"topic": "온디바이스 AI 반도체"}
)

print(result)

```

전체 구조는 다음과 같다.



29. AutoGen 전체 예제

기본적인 대화형 Agent는 다음과 같이 구성할 수 있다.

Python

실행됨

```
import os
import autogen

llm_config = {
    "config_list": [
        {
            "model": "gpt-4o-mini",
            "api_key": os.environ["OPENAI_API_KEY"]
        }
    ],
    "temperature": 0,
}

assistant = autogen.AssistantAgent(
    name="Researcher",
    system_message=(
        "너는 시장 조사 전문가다. "
        "요청받은 주제의 핵심 트렌드를 간결하게 정리한다."
    ),
    llm_config=llm_config,
)

user_proxy = autogen.UserProxyAgent(
    name="User",
    human_input_mode="NEVER",
    code_execution_config=False,
    max_consecutive_auto_reply=1,
)

user_proxy.initiate_chat(
    assistant,
    message="온디바이스 AI 반도체 트렌드 3가지를 알려줘."
)
```

개념적으로 다음과 같다.

User

|

| 질문

▼

Researcher



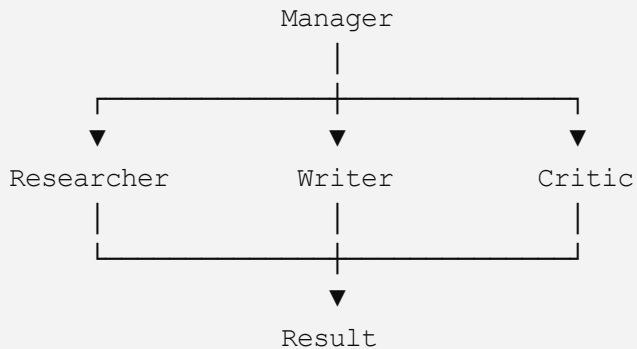
답변



User

30. 다중 Agent 실습 구조

교육 과정에서는 다음과 같은 실습이 적합하다.



동일한 문제를 두 프레임워크로 구현한다.

CrewAI

Agent



Task



Crew



Process

AutoGen

Agent



GroupChat



GroupChatManager

이를 통해 단순히 문법을 배우는 것이 아니라 "멀티에이전트 협업을 어떤 추상화 방식으로 구현하는가"를 비교할 수 있다.

31. 실습 과제 제안

주제는 "온디바이스 AI 반도체 시장 분석 보고서 생성"으로 구성할 수 있다.

Agent 구성

Researcher

→ 시장 및 기술 동향 조사

Writer

→ 조사 결과를 보고서로 작성

Critic

→ 보고서의 오류와 부족한 내용을 검토

CrewAI 실습

1. `Process.sequential` 로 구현한다.
2. Researcher → Writer 순서로 실행한다.
3. Reviewer를 추가한다.
4. `Process.hierarchical` 로 변경한다.
5. Manager가 Agent의 작업을 조정하도록 구성한다.

AutoGen 실습

1. Researcher를 만든다.
2. Writer를 추가한다.
3. Critic을 추가한다.
4. GroupChat을 구성한다.
5. `speaker_selection_method="auto"` 를 사용한다.
6. Critic이 문제가 있다고 판단하면 Writer가 다시 작업하도록 구성한다.
7. `max_round` 를 이용하여 무한 반복을 방지한다.

32. 핵심 정리

CrewAI와 AutoGen은 멀티에이전트 시스템을 높은 수준에서 구현하기 위한 프레임워크이다.

CrewAI는 다음과 같이 이해하면 된다.

역할

↓

Agent

↓

Task

↓

Crew

↓

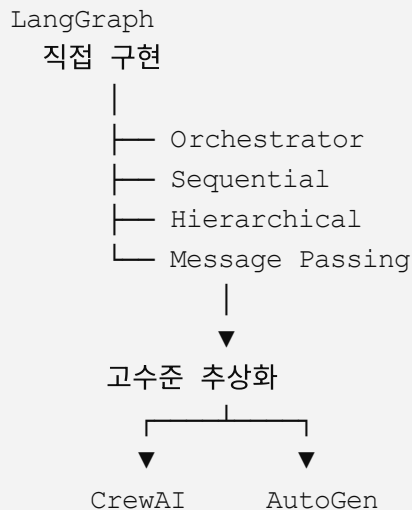
즉, "각 Agent에게 어떤 역할과 작업을 맡길 것인가"가 핵심이다.

AutoGen은 다음과 같이 이해하면 된다.



즉, "Agent들이 어떻게 대화하고 협업할 것인가"가 핵심이다.

앞에서 학습한 LangGraph와 연결하면 다음과 같이 정리할 수 있다.



따라서 CrewAI와 AutoGen은 LangGraph와 완전히 다른 개념이라기보다는, 앞에서 학습한 멀티에이전트 설계 패턴을 더 간단하게 표현할 수 있도록 추상화한 프레임워크로 이해하는 것이 가장 적절하다.

특히 CrewAI는 "역할 기반 협업", AutoGen은 "대화 기반 협업"이라는 차이를 중심으로 이해하면 이후 실습에서 두 프레임워크의 차이를 쉽게 구분할 수 있다.